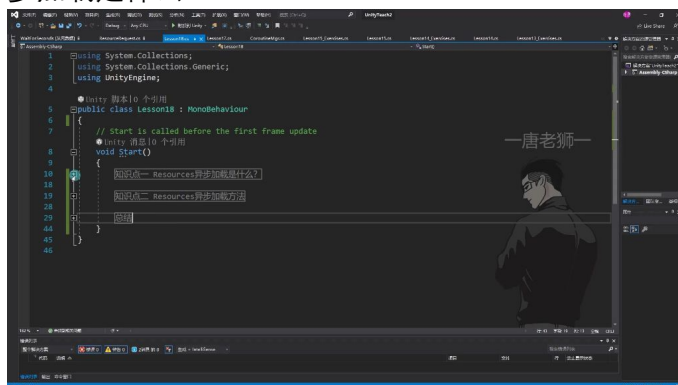
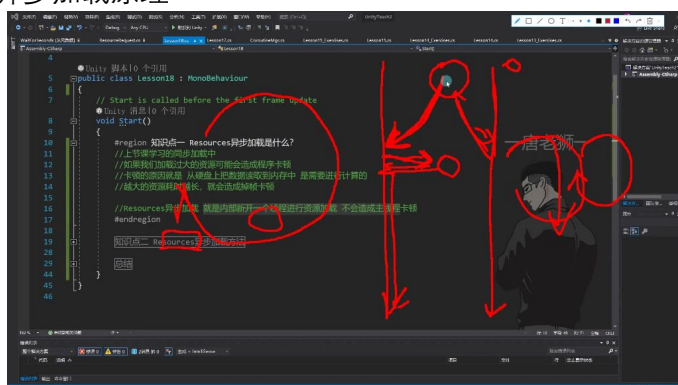


一、资源异步加载 00:08

1. 异步加载是什么 00:29



- 1) 同步加载的问题
 - 卡顿原因: 从硬盘读取数据到内存需要计算, 资源越大耗时越长, 导致掉帧卡顿。例如加载30MB资源时, 耗时可能超过单帧时间(60帧游戏每帧约16.66ms)
 - 表现现象: 同步加载会阻塞主线程, 必须完全加载完成后才能继续执行后续逻辑, 导致帧率下降(如从60帧降至30帧)
- 2) 异步加载原理

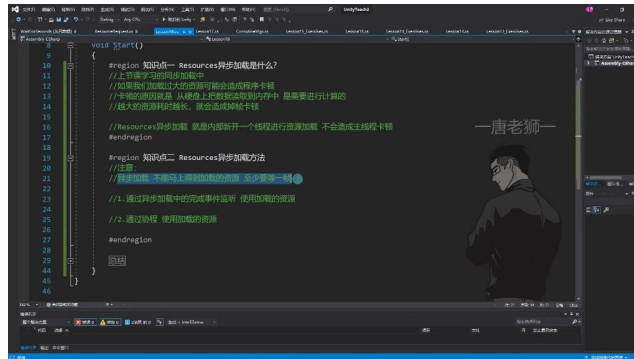


- 多线程机制: 内部新开独立线程专门加载资源, 主线程继续执行其他逻辑
- 数据交互: 通过公共内存区域传递加载完成的资源, 避免多线程直接访问Unity主线程对象
- 优缺点对比:
 - 优点: 彻底解决主线程卡顿问题
 - 缺点: 不能立即使用资源, 必须等待加载完成
- 3) 实现特点
 - 延迟使用: 至少需要等待一帧后才能获取加载完成的资源
 - 线程安全: 虽然Unity限制多线程直接操作引擎对象, 但通过中间存储区实现安全交互
 - 适用场景: 特别适合加载大型资源(如场景、高清贴图、视频等)

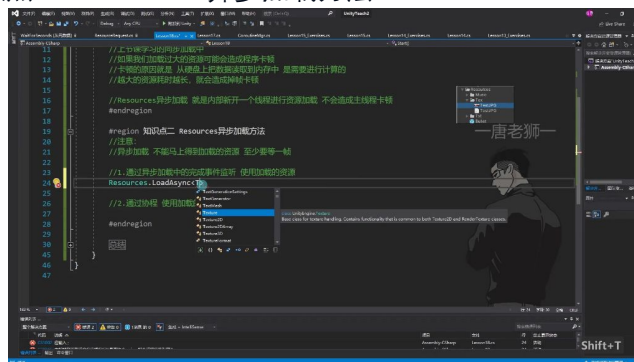
2. 异步加载方法 05:31

1) 通过异步加载中的完成事件监听使用资源 06:07

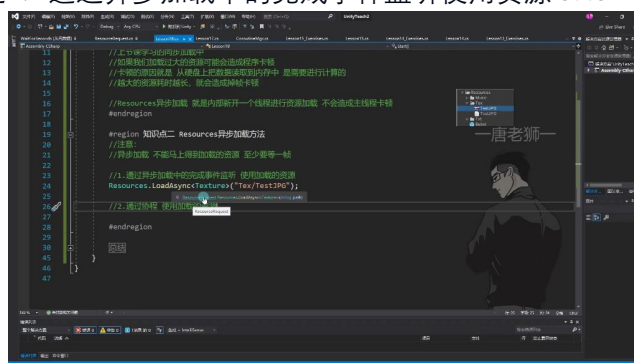
- 知识点一: Resources异步加载是什么? 06:17



-
- **同步加载问题：**上节课学习的同步加载中，如果加载过大的资源可能会造成程序卡顿。卡顿的原因是从硬盘读取数据到内存需要进行计算，资源越大耗时越长，会造成掉帧卡顿。
- **异步加载优势：**Resources异步加载是内部新开一个线程进行资源加载，不会造成主线程卡顿。主线程继续执行其他逻辑，资源加载完成后通过回调通知主线程。
- **知识点二：Resources异步加载方法 06:27**



-
- **加载特性：**
 - 异步加载不能立即得到资源，至少需要等待一帧
 - 有两种使用方法：完成事件监听和协程方式
- **API说明：**
 - 使用Resources.LoadAsync<T>方法进行异步加载
 - 该方法有泛型和非泛型两种重载，推荐使用泛型方法避免同名资源冲突
- **例题1：通过异步加载中的完成事件监听使用资源 07:01**



-
- **实现步骤：**
 - 调用Resources.LoadAsync<Texture>("Tex/TestJPG")开始异步加载
 - 获取返回的ResourceRequest对象
 - 通过completed事件监听加载完成
 - 在回调函数中处理加载完成的资源
- **关键代码：**

```
1 ResourceRequest rq = Resources.LoadAsync<Texture>("Tex/TestJPG");
2 rq.completed += LoadOver;
```

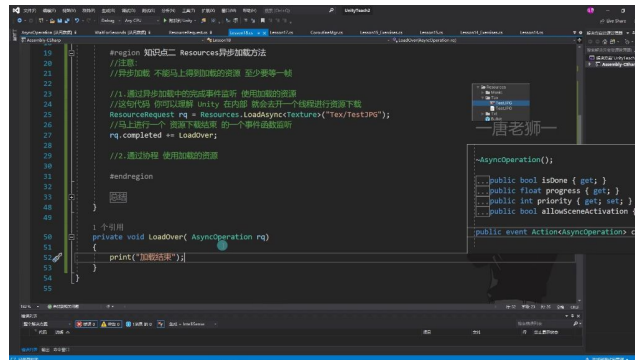
```

3
4 private void LoadOver(AsyncOperation rq) {
5     tex = (rq as ResourceRequest).asset as Texture;
6     print("加载结束");
7 }

```

- 注意事项:

- 不能直接在加载语句后立即使用资源，必须等待加载完成
- 回调函数的参数类型必须匹配 `Action<AsyncOperation>` 委托
- 需要通过类型转换获取具体资源类型

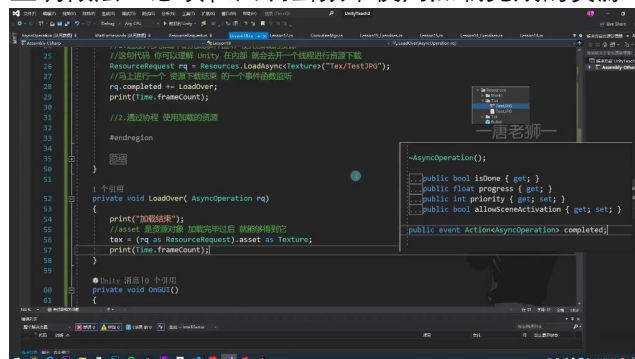


- 内部机制:

- Unity内部会创建新线程进行资源加载
- 主线程继续执行其他逻辑
- 加载完成后自动调用注册的回调函数
- 通过 `ResourceRequest.asset` 获取加载完成的资源

- 常见错误:

- 错误: 直接在加载语句后使用 `rq.asset`
- 原因: 资源尚未加载完成, 此时 `asset` 为 `null`
- 正确做法: 必须在回调函数中使用加载完成的资源

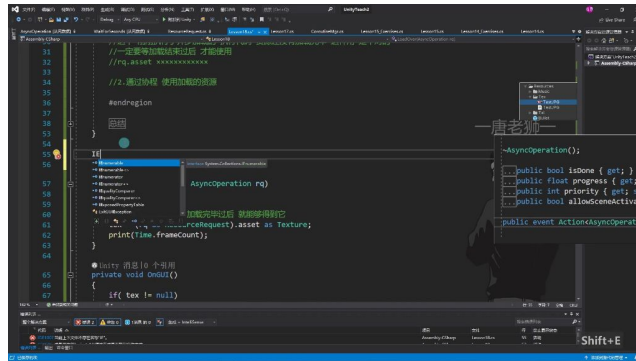


- 时序验证:

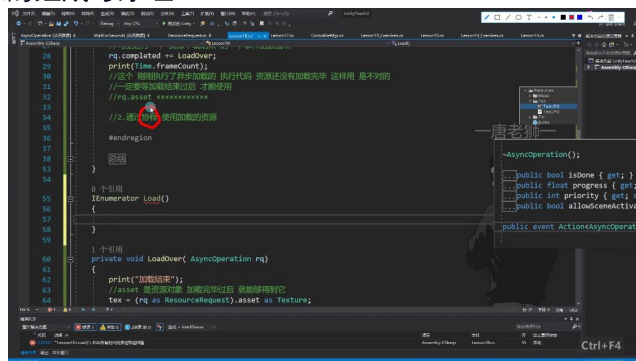
- 在开始加载时打印当前帧数: `print(Time.frameCount)`
- 在回调函数中再次打印帧数
- 两次数值至少相差1, 证明异步加载至少需要等待一帧

2) 通过协程使用资源 16:48

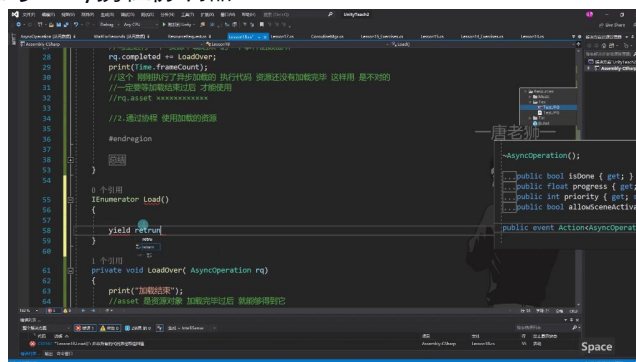
- 协程与资源加载 16:51



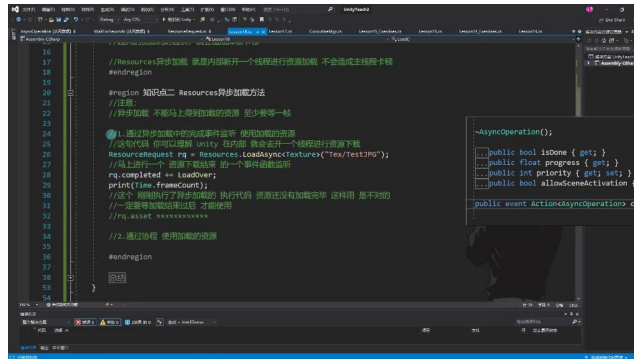
- 协程函数要求：必须声明为迭代器接口返回类型，函数名可自定义（如示例中的Load函数）
- 异步加载原理：Resources.LoadAsync方法内部会新开线程进行资源加载，避免主线程卡顿
- 协程的组成与原理 17:11



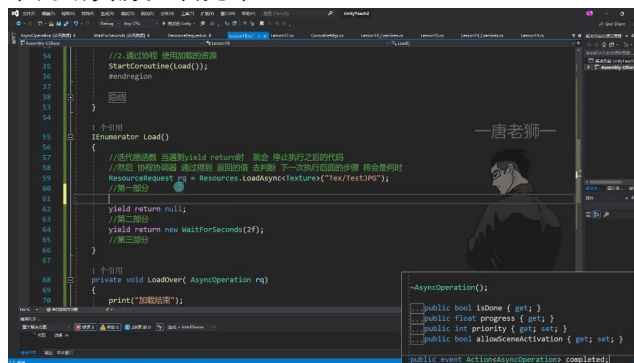
- 两大组成部分：
 - 迭代器函数：通过yield return实现分步执行
 - 携程协调器：Unity内置在MonoBehaviour中的调度系统
- 执行特点：遇到yield return会暂停执行，协调器根据返回值决定后续执行时机
- 协程与Unity携程协调器 17:53



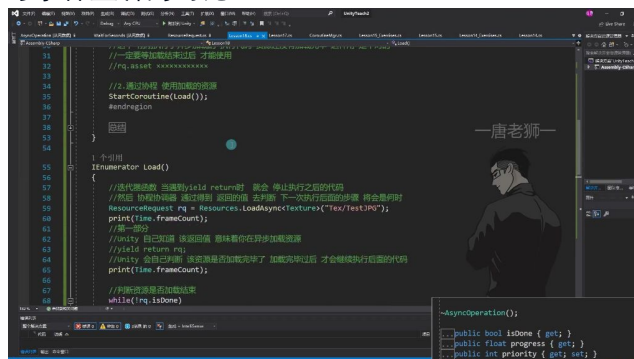
- 返回值类型：
 - yield return null：下一帧继续执行
 - yield return new WaitForSeconds(2f)：等待2秒后继续
- 分段执行：每个yield return将代码逻辑分成多个部分依次执行
- 异步加载资源的方法 19:38



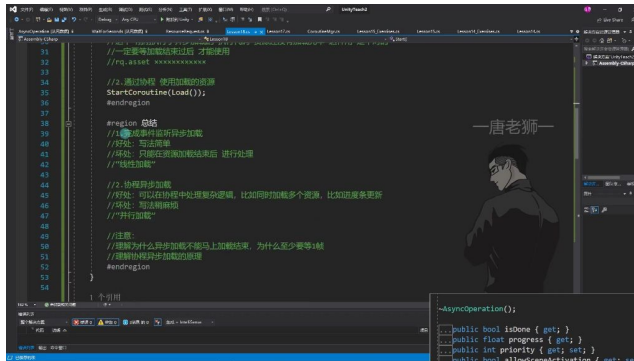
- 核心方法：ResourceRequest rq
Resources.LoadAsync<Texture>("Tex/TestJPG") =
- 注意事项：
 - 不能立即获取资源，至少需要等待一帧
 - 资源加载进度通过rq.progress获取（0-1范围）
- 协程中判断资源加载完毕 20:41



- 自动判断：直接yield return rq时Unity会自动检测加载完成状态
- 手动检测：
 - 基类关系：AsyncOperation和WaitForSeconds都继承自YieldInstruction
- 协程与事件监听的对比 27:21



- 事件监听方式：
 - 优点：写法简单
 - 缺点：只能在加载完成后处理，无法中途干预
- 协程方式：
 - 优点：可处理复杂逻辑（如多资源并行加载、进度更新）
 - 缺点：代码结构稍复杂
- 协程异步加载的总结 27:36



-
- **关键理解：**
 - 异步加载为什么不能立即完成（至少需要一帧）
 - 协程异步加载的内部原理（Unity自动检测机制）
- **应用场景：**
 - 简单加载使用事件监听
 - 复杂逻辑使用协程方式

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
Resources异步加载的概念	通过新开线程加载资源，避免主线程卡顿，但资源需加载完成后才能使用。	同步加载与异步加载的 本质区别 （主线程阻塞 vs 后台线程加载）。异步加载 不能立即使用资源 ，需等待完成。	★★★
异步加载的实现方法（事件监听）	使用Resources.LoadAsync并监听completed事件，在回调函数中处理加载完成的资源。	错误用法： 直接在同帧访问asset（资源未加载完成）。 正确逻辑： 通过事件回调确保资源就绪。	★★
异步加载的实现方法（协程）	在协程中返回ResourceRequest对象，Unity自动判断加载完成状态；可结合isDone和progress实现进度监控。	协程分阶段执行： yield return控制加载等待。 多资源并行处理优势： 协程更适合复杂逻辑（如同时加	★★★★

		载多个资源并拼接)。	
同步 vs 异步加载对比	同步加载导致主线程卡顿 (大资源耗时 > 16.6ms会掉帧)，异步加载通过线程分离解决卡顿。	性能影响： 同步加载的帧时间计算。 异步缺点： 资源延迟可用性。	★ ★ ★
协程与事件监听的适用场景	事件监听：简单单资源加载。 协程： 多资源协作 、进度条更新等复杂逻辑。	设计选择： 线性任务 vs 并行任务需求。	★ ★ ★